



Universidad de Costa Rica

Escuela de Ciencias de la Computacion e Informatica
CI-0123 Proyecto Integrador de Sistemas Operativos y Redes

Diseño System File

Estudiantes:

Jose Linarte Sandino C4L627
Keilyn Vega Aguilar C4K849
Sasha Chavarría Arias C32078
Andrey Bejarano Díaz C31070

Docentes:

Amanda Alpízar
Francisco Arroyo

II 2026

Índice

Introducción.....	2
Superbloque.....	2
Bitmap de Bloques.....	3
Organización de Bloques.....	3
Encadenamiento de Bloques.....	3
Operaciones para el manejo del SystemFile.....	4
Ventajas del Diseño.....	5
Desventajas.....	5
Conclusión.....	5

Introducción

Un sistema de archivos se encarga de permitir el acceso a los datos y organizarlos dentro de dispositivos de almacenamiento. El siguiente texto explica cómo funcionará nuestro sistema de archivos.

El sistema propuesto utiliza bloques de tamaño fijo de 256 bytes, un superbloque que centraliza los metadatos generales del archivo (firma de identificación, cantidad total de bloques y punteros a las estructuras principales), un bitmap de asignación, bloques de índice, y un mecanismo de encadenamiento mediante punteros (`siguiente_bloque`) que permite almacenar registros cuyo contenido excede el tamaño de un solo bloque, enlazando bloques adicionales de la misma forma en que lo hace una lista enlazada.

Firma del Sistema de Archivos

Todo archivo de nuestro sistema de almacenamiento comienza con una firma de identificación, ubicada en los primeros 4 bytes del bloque 0 (el superbloque). Esta firma es una secuencia fija de caracteres ("KEY1") que se escribe una única vez al momento de crear el archivo y que se verifica cada vez que el programa lo abre.

La idea es garantizar la compatibilidad del archivo, si los primeros 4 bytes no coinciden con la firma esperada es porque el archivo no pertenece al sistema de almacenamiento (está corrupto, otro formato o fue generado por otro programa).

Superbloque

Este bloque contiene los metadatos generales que describen la estructura completa del sistema de almacenamiento.

- Firma del sistema
- Tamaño del bloque
- Cantidad de bloques

- Puntero al bitmap
- Puntero al índice de categorías
- Puntero al índice de órdenes

Bitmap de Bloques

Esta estructura es esencial para indicar que bloques se encuentran libres u ocupados.

- Bloque libre = 0
- Bloque ocupado = 1

Ej:

Bloque#: 0 1 2 3
 Bit: 1 1 1 0

Organización de Bloques

Bloque	Contenido
0	Superbloque — metadatos generales del archivo
1	Bitmap de asignación — control de bloques libres/ocupados
2 en adelante	Índice de categorías
continuación	Bloques de categoría — contienen los productos embebidos de cada categoría
continuación	Índice de órdenes
continuación	Bloques de orden — contienen los productos solicitados por el cliente en cada compra

Encadenamiento de Bloques

Como cada bloque tiene un tamaño fijo, es posible que la información de una categoría no alcance toda en un único bloque.

Para resolver lo anterior el sistema usa un sistema de encadenamiento por medio de punteros. Cada bloque reserva un campo (`siguiente_bloque`) que indica el número de bloque donde continúa la misma estructura lógica.

Existen dos casos, cuando la información cabe completa en un solo bloque, se establece un valor reservado que indica el fin de la cadena (0xFFFFFFFF). Cuando la información no cabe, este campo contiene el número del siguiente bloque que puede encontrarse en cualquier posición del archivo.

Bloque 12:

[id_categoria=3][siguiente_bloque=45][cant_productos=7][producto1]...[producto7]

Bloque 45:

[id_categoria=3][siguiente_bloque=0xFFFFFFFF][cant_productos=3][producto8]...[producto10]

Operaciones para el manejo del SystemFile

Para crear un archivo de almacenamiento cuando aún no existe, el sistema debe inicializarse antes de poder usarse. Este proceso consiste en:

1. Crear el archivo en disco con un tamaño inicial reservado (por ejemplo, N bloques $\times$ 256 bytes).
2. Escribir el bloque 0 (superbloque), incluyendo la firma del sistema, el tamaño de bloque, la cantidad total de bloques, y los punteros al bitmap y a los índices (inicialmente apuntando a bloques vacíos, ya que aún no hay categorías ni órdenes).
3. Escribir el bloque 1 (bitmap), marcando como ocupados únicamente los bloques ya utilizados por el superbloque, el propio bitmap y los bloques iniciales de los índices; el resto se marca como libre.
4. Escribir los bloques iniciales de los índices de categorías y de órdenes, vacíos (sin entradas todavía).

Para leer una categoría completa, el programa sigue este procedimiento:

1. Consulta el índice de categorías para obtener el bloque inicial.
2. Lee ese bloque y procesa sus productos.
3. Revisa el campo siguiente_bloque: si es distinto del valor reservado, salta a ese bloque y repite el proceso.
4. Se detiene cuando encuentra el valor reservado, indicando que no hay más bloques encadenados.

Escritura de una nueva categoría (o producto dentro de una categoría existente)

1. Consultar el índice de categorías para verificar si la categoría ya existe.
 - Si no existe: buscar en el bitmap el primer bloque libre, marcarlo como ocupado, escribir ahí el nuevo bloque de categoría (con el producto correspondiente) y agregar una nueva entrada al índice de categorías apuntando a ese bloque.
 - Si ya existe: seguir la cadena de bloques de esa categoría (usando siguiente_bloque) hasta encontrar un bloque con espacio disponible.
2. Si el último bloque de la cadena está lleno (no caben más productos), buscar un nuevo bloque libre en el bitmap, escribir ahí el producto, y actualizar el campo siguiente_bloque del bloque anterior para que apunte al nuevo bloque.
3. Actualizar el bitmap para reflejar el nuevo bloque como ocupado.
4. Guardar los cambios en disco (write(ptr, tam) seguido de flush() o cierre del archivo, según corresponda).

Escritura de una nueva orden

1. Buscar en el bitmap el primer bloque libre y marcarlo como ocupado.
2. Escribir en ese bloque el encabezado de la orden (id_orden, siguiente_bloque = 0xFFFFFFFF, cantidad_productos) seguido de los productos solicitados por el cliente.
3. Si la cantidad de productos de la orden excede lo que cabe en un bloque, repetir el proceso de encadenamiento descrito en la sección anterior.
4. Agregar una nueva entrada al índice de órdenes, apuntando al bloque inicial de la orden recién creada.
5. Actualizar el bitmap y guardar los cambios en disco.

Ventajas del Diseño

El diseño propuesto permite un acceso directo y eficiente a los datos mediante bloques e índices de tamaño fijo, evitando recorrer el archivo completo para localizar una categoría o una orden. El mecanismo de encadenamiento permite que cada registro crezca dinámicamente según lo necesite, sin desperdiciar espacio ni imponer un límite artificial de tamaño, mientras que el bitmap facilita la reutilización eficiente de los bloques liberados.

Desventajas

Una desventaja específica de este diseño es que, al almacenar varios productos dentro de un mismo bloque de categoría, cualquier cambio en un solo producto obliga a leer y reescribir el bloque completo, incluyendo los demás productos que no sufrieron ningún cambio. Esto ocurre porque el sistema de archivos solo permite leer o escribir bloques completos, no partes individuales dentro de ellos. Como consecuencia, productos con muchos cambios por ejemplo con mucho stock generan actualizaciones de disco innecesariamente costosas, ya que se reescribe información que no cambió.

Conclusión

El diseño propuesto logra un balance razonable entre simplicidad y funcionalidad, ofreciendo acceso eficiente a categorías y órdenes dentro de las restricciones de bloques de 256 bytes.